Rust DSL开发文档

类的定义

在 `classes::classes!` 宏内，使用下列语法定义一个类：

代码块

```
classs::classes! {
    // 定义Person类
    pub class Person {
        // Person类的数据字段
        struct {
            // 定义一个不可变的字段`name`，数据类型为`String`
            final name: String,
        }

        // 定义Person类的构造函数
        pub fn new(name: String) -> Self {
            Self { name }
        }

        // 定义Person类的实例方法
        pub fn greet(&self) {
            // 通过`self.get_name()`访问字段
            println!("Hello, my name is {}", self.get_name());
        }
    }
}

fn main() {
    // 调用Person类的构造函数构造实例，构造出来的实例位于堆中，
    // 其类型为引用计数的胖指针`CRc<Person>`类型
    let person /* : CRc<Person> */ = Person::new("张三".to_string());
    // 调用person的实例方法
    person.greet(); // 输出: Hello, my name is 张三
}
```

其中，`struct { }` 为类的数据部分，类似于Rust `struct`。剩余的各个 `fn` 为类的行为部分，分为三个类别：

- 构造函数，用于初始化类的数据，也用于构造类的实例。其特征是：不含 `&self` 参数，返回 `Self` 类型（不能返回类名 `ClassName` 否则视为普通类函数）。

- 实例方法，用于实现类对象实例的行为，通过继承与覆写实现多态。其特征是：含 `&self` 参数。
- 类函数，用于实现类的行为，不支持多态。其特征是：不含 `&self` 参数，且返回值不为 `Self` 。

字段定义与访问

在 `class` 内，使用 `struct { }` 块定义字段（其中 `struct { }` 块只能位于 `class` 的头部）。

由于 `class` 大部分情况下（目前是全部情况）都在引用计数的 `CRc` （即Class `Rc` 的简称）类型中使用，因此与通常的Rust `struct` 中的字段不同，字段默认是内部可变的（即可以通过共享引用 `&self` 实现字段的修改），但允许使用**字段修饰符（modifier）**改变其可变性。

字段的可变性

按字段类型及其读写频率，可以分为下列几种情况：

字段类型		读写需求		
		只读	读写	只允许写一次
`T: Copy`		`final x: T`	`x: T`	`late final x: T`
`C: Class`	强引用	`final x: CRc<C>` `final x: Option<CRc<C>>`	`x: CRc<C>` `x: Option<CRc<C>>`	`late final x: CRc<C>` `late final x: Option<CRc<C>>`
	弱引用	`final x: CWeak<C>` `final x: Option<CWeak<C>>`	`x: CWeak<C>` `x: Option<CWeak<C>>`	`late final x: CWeak<C>` `late final x: Option<CWeak<C>>`
`Rc<T>` 或其他浅拷贝类型 `U`		`final x: Rc<T>` `final x: RcLike<U>`	`x: Rc<T>` `x: RcLike<U>`	`late final x: Rc<T>` `late final x: RcLike<U>`
其他类型（如 `Vec` , `HashMap` 等）		`final x: T`	`mutable x: T`	`late final x: T`

- `Copy` 的类型（如 `bool` , `usize` 等），使用 `get` , `set` 按值读写。其行为等同于 `core::cell::Cell` 。
- `Class` 类型，无论强弱引用，都使用 `get` , `set` 按值读写（其中，弱引用在读时自动upgrade，在写时自动downgrade，因此强弱引用的读写都通过强引用进行）。其行为等同于 `core::cell::Cell` （读时发生 `clone` ，强引用计数加一）。

- `Rc` 类型与其他浅拷贝类型，与 `Class` 类型的使用方式类似。
- 其他类型，必须使用 `mutable` 修饰符表示可变，或用 `final` 表示不可变。使用 `get` 获取共享引用，使用 `get_mut` 获取可变引用，通过 `set` 按值写入。其行为等同于 `core::cell::RefCell`。
- 加 `final` 修饰后的字段为只读字段，使用 `get` 获取可变引用。
- 加 `late final` 修饰的字段只允许写入一次。使用 `get` 获取可变引用，使用 `set` 按值写入。试图 `get` 未 `set` 的字段，或多次 `set` 均会导致panic。

> ❗ 注意：由于实现方式的限制，当使用标准库以外的 `Clone` 类型时，需要为其实现 `classes::get_set::GetSet` trait，并使用 `classes::prelude::RcLike` 包装（与原类型无异，只是为了让过程宏知道这是一个 `Clone` 类型），方法是调用 `classes::impl_get_set_for_clone_type!(unsafe T)`，否则会报 `` `T` is not `Copy`, consider adding a `final` or a `mutable` modifier to the field ``

如下所示为字段可变性的使用示例：

代码块

```
classes! {
    pub class Teacher {
        struct {
            // 定义不可变字段
            final id: u64,
            final name: String,
            // 定义只允许写入一次的字段
            late final subject: String,
            // 定义可变字段
            mutable students: Vec<CRc<Student>>,
        }
        pub fn new(id: u64, name: String) -> Self {
            // 使用`..`省略`late final`字段的值
            Self { id, name, students: Vec::new(), .. }
        }

        // 按引用读取不可变字段
        pub fn name(&self) -> &str { self.get_name() }
        // 按值读取不可变字段
        pub fn id(&self) -> u64 { self.get_id() }
        pub fn teaching_students(&self) -> Vec<&str> {
            // 按引用读取可变字段
            self.get_students().iter().map(|stu| stu.name()).collect()
        }
```

```rust
        pub fn add_student(&self, student: CRc<Student>) {
            student.change_teacher(self.clone());
            // 按引用修改可变字段
            self.get_mut_students().push(student);
        }
        pub fn clear_students(&self) {
            // 按值写入可变字段
            self.set_students(Vec::new());
        }
    }
    pub class Student {
        struct {
            // 定义不可变字段
            final id: u64,
            final name: String,
            // 定义可变字段
            teacher: Option<CWeak<Teacher>>,
        }
        pub fn new(id: u64, name: String) -> Self {
            Self { id, name, teacher: None }
        }

        // 按引用读取不可变字段
        pub fn name(&self) -> &str { self.get_name() }
        // 按值读取不可变字段
        pub fn id(&self) -> u64 { self.get_id() }
        pub fn change_teacher(&self, teacher: CRc<Teacher>) {
            // 按值写入可变字段（自动downgrade，自动加Some）
            self.set_teacher(teacher);
        }
        pub fn teacher(&self) -> Option<CRc<Teacher>> {
            // 按值读取可变字段（自动upgrade）
            self.get_teacher()
        }
    }
}

fn main() {
    let teacher = Teacher::new(0, "张三");

    // `final`字段，只读
    // ==================================
    // teacher.set_id(1); // Compile Error: no such method `set_id`
    // 输出: Teacher 张三's ID is 0
    println!("Teacher {}'s ID is {}", teacher.get_name(), teacher.id());

    // `late final`字段，只允许写入一次，写入前不允许读取
```

```
    // ==================================
    // Panic: `late final` field not initialized
    // println!("Teacher {} teaches {}", teacher.get_name(), teacher.get_subject());
    teacher.set_subject("语文");
    // teacher.set_subject("数学"); // Panic: `late final` field already initialized
    // 输出: Teacher 张三 teaches 语文
    println!("Teacher {} teaches {}", teacher.get_name(), teacher.get_subject());

    let student1 = Student::new(1, "李四");
    let student2 = Student::new(2, "王五");

    // `mutable`字段，允许按引用读写
    // ==================================
    teacher.add_student(student1.clone());
    teacher.add_student(student2.clone();
    // 输出: Teacher 张三 now teaches students ["李四", "王五"]
    println!("Teacher {} now teahces students {:?}", teacher.teaching_students());
}
```

字段的默认值

在定义字段时，可以指定一个默认值，于是在构造函数中便可省略该字段。

如上述的类，使用默认值的写法如下：

代码块

```
classes! {
    pub class Teacher {
        struct {
            final id: u64,
            final name: String,
            late final subject: String,
            // `late`或`late final`的默认值中支持使用`self`
            late foo: u32 = self.create_foo(),
            // students的默认值为`Vec::new()`
            mutable students: Vec<CRc<Student>> = Vec::new(),
        }
        pub fn new(id: u64, name: String) -> Self {
            // 构造函数中可以使用`..`省略`students`，使用默认值`Vec::new()`
            Self { id, name, .. }
        }
        fn create_foo(&self) -> u32 { 0 }
```

```
        // ...
    }
    pub class Student {
        struct {
            final id: u64,
            final name: String,
            teacher: Option<CWeak<Teacher>> = None,
        }
        pub fn new(id: u64, name: String) -> Self {
            // `teacher`使用默认值`None`
            Self { id, name, .. }
        }

        // ...
    }
}
```

> ❗ 暂不支持Dart的可选参数形式带默认值的形式，有打算使用Rust中的Builder模式构造来实现Dart的可选参数，但现在只能使用显示在函数将所有可选参数作为必选参数，并用注释标上默认参数

继承与多态

Rust DSL中支持单继承（通过 `extends ClassName` 表示）。当多个类有共同成员字段与方法时，可以使用继承来抽象；共享同样成员字段的不同类之间行为有差异时，可以使用覆写（override）实现多态。

如上述示例中，可以将 `Teacher` 与 `Student` 的共同字段 `id` 与 `name` 抽象成一个公共父类。

代码块

```
classes! {
    // 定义公共父类
    class Person {
        struct {
            final id: u64,
            final name: String,
        }

        pub fn new(id: u64, name: String) -> Self {
            Self { id, name }
        }
    }
    // 定义`Teacher`类继承自`Person`类
```

```
    class Teacher extends Person {
        struct {
            late final Subject: String,
            mutable students: Vec<CRc<Student>> = Vec::new(),
        }
        pub fn new(id: u64, name: String) -> Self {
            // 使用`super: Super::new(id, name)`调用父类的构造函数
            // 注意此处必须使用`Super`作为父类，不能使用`Person::new`。
            // 因为`Person::new`将构造出一个`CRc<Person>`而不是
            // `Person`类的数据部分
            Self { super: Super::new(id, name), .. }
        }
        // ...
    }
    class Student extends Person {
        struct {
            teacher: Option<CWeak<Teacher>> = None,
        }
        pub fn new(id: u64, name: String) -> Self {
            // 也可以使用`let super = Super::new`调用父类的构造函数。
            // 该写法可用于调整父类构造函数与
            let super = Super::new(id, name);
            Self { super, .. }
        }
    }
}
```

实例方法覆写（override）

继承关系中的子类允许覆写父类的实例方法（在函数声明时使用 `override` ），以实现同一父类在运行时根据不同的子类表现不同行为的能力，即多态。

如下所示为一个覆写的示例：

代码块

```
classes! {
    class Animal {
        pub fn new() -> Self { Self {} }
        pub fn eat(&self) {
            println!("Animal::eat");
        }
    }

    class Dog extends Animal {
        pub fn new() -> Self {
            Self { super: Super::new() }
```

```
        }

        // 使用`override`覆写父类的方法eat
        pub override fn eat(&self) {
            println!("Dog::eat");
        }
    }
}

fn main() {
    let animal = Animal::new();
    animal.eat(); // 输出: Animal::eat
    let dog = Dog::new();
    dog.eat(); // 输出: Dog::eat
}
```

❗ 注意子类覆写父类函数时必须使用 `override` ，否则子类仅定义了一个与父类同名的函数，且无任何编译错误提示！

抽象类

有时，公共父类通过抽象，无法提供具体的实例方法实现，这时可以将其声明为抽象类（使用 `abstract class` 声明）。如上述用例中的 `Animal` 类可以被声明为抽象类：

代码块

```
classes! {
    // 使用`abstract class`声明一个抽象类
    abstract class Animal {
        pub fn new() -> Self { Self {} }
        // 抽象类中，允许有未实现的方法，也称为纯虚方法。
        pub fn eat(&self);
    }

    class Dog extends Animal {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }

        // 非抽象类必须实现父类中的所有纯虚方法。
        pub override fn eat(&self) {
            println!("Dog::eat");
        }
    }
}
```

```

fn main() {
    // 抽象类不允许构造实例
    // Compile Error: no such method
    // let animal = Animal::new();

    // 非抽象类允许构造实例
    let dog = Dog::new();
    dog.eat(); // 输出: Dog::eat
}
```

多级继承

对于多级继承的情况，在覆写祖类方法时，需要指定祖类限定名 `override fn Superclass::method` ，如下所示：

代码块

```
classes! {
    class Alice {
        pub fn new() -> Self { Self {} }

        pub fn eat(&self) {
            println!("Alice::eat");
        }
    }

    class Bob extends Alice {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }

        // 覆写直接父类的方法，父类限定名可省略。
        pub override fn eat(&self) {
            println!("Bob::eat");
        }
    }

    class Carol extends Bob {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }

        // 覆写间接祖类的方法（指定义在祖类中的方法），祖类限定名不能省略
        pub override fn Alice::eat(&self) {
            println!("Carol::eat");
```

```
        }
    }
}

fn main() {
    let carol = Carol::new();
    carol.eat(); // 输出: Carol::eat
    let bob = Bob::new();
    bob.eat(); // 输出: Bob::eat
    let alice = Alice::new();
    alice.eat(); // 输出: alice::eat
}
```

有继承时的子类/父类构造顺序

Rust DSL中构造函数的执行顺序是：先构造子类字段，再构造父类字段，然后执行父类初始化，最后执行子类初始化。

代码块

```
classes! {
    class Extend extends Base {
        struct {
            x: u32,
            y: u32,
            z: u32,
        }
        pub fn new() -> Self {
            // 先构造子类字段，再构造父类字段
            let x = 0;
            let y = 0;
            let z = 0;
            // 注意: 20250703之后的版本，不再支持以下写法
            // let super = Super::new();
            Self { super: Super::new(), x, y, z }
        }
    }
}
```

❗ 注意：构造函数若要使用 `self` 或调用任何类的实例方法，即当Dart中构造函数使用到 `this` 时（读、写、调用成员方法），需要使用下列特殊语法。

代码块

代码块

```dart
class Base {
    int x;
    Base() : this.x = 0, {
        foo();
    }

    void foo() => print('Base.foo');
}

class Derived extends Base {
    Derived() : super() {
        foo();
    }

    @Override
    void foo() => print('Derived.foo');
}

void main() {
    Base derived = Derived();
}
// 输出:
// Derived.foo
// Derived.foo
```

```rust
classes! {
    class Base {
        struct {
            x: i32,
        }
        pub fn new() -> Self {
            // `let self`之前,
            // 对应Dart构造函数参数列表
            // 不允许使用`self`
            let self = Self { x: 0 };
            // `let self`之后,
            // 对应Dart构造函数体,
            // 可以使用`self`
            self.foo();
            self
        }
        pub fn foo(&self) {
            println!("Base::foo");
        }
    }
    class Derived extends Base {
        pub fn new() -> Self {
            let self = Self {
                super: Super::new(),
            };
            self.foo();
            self
        }
        pub override fn foo(&self) {
            println!("Derived::foo");
        }
    }
}

fn main() {
    let derived: CRc<Base> =

Derived::new().into_super();
}
// 输出:
// Derived::foo
```

```
// Derived::foo
```

接口与实现

类除了被继承以外，还允许被作为接口实现（使用 `implements ClassName` 表示实现接口类 `ClassName` ）。当作为接口实现时，不论该方法是否为纯虚方法，**子接口类都必须覆写父接口类的所有方法**。

如下所示为接口的使用方法：

代码块

```
classes! {
    // 推荐使用抽象类定义接口
    abstract class Bark {
        pub fn bark(&self);
    }

    class Dog implements Bark {
        pub fn new() -> Self { Self {} }

        // 覆写父接口的实例方法
        pub override fn bark(&self) {
            println!("Dog::bark");
        }
    }
}

fn main() {
    let dog = Dog::new();
    dog.bark(); // 输出: Dog::bark
}
```

❗ 注意：子接口类实现父接口类时，并不会继承父接口类的数据，因此子接口类不能访问父接口类的任何字段。

代码块

```
classes! {
    // Bark类作为接口使用时，等效于一个不含任何字段与方法实现的抽象类:
    /*
    abstract class Bark {
        pub fn bark(&self);
    }
```

```
    */
    class Bark {
        struct {
            final sound: String,
        }
        pub fn bark(&self) {
            println!("Bark::bark {}", self.get_sound());
        }
    }

    class Dog implements Bark {
        pub fn new() -> Self { Self {} }

        // 覆写父接口的实例方法
        pub override fn bark(&self) {
            println!("Dog::bark");
            // 不能使用父接口类中的字段
            // Compile Error: no such method
            // self.get_sound()
        }
    }
}

fn main() {
    let dog = Dog::new();
    dog.bark(); // 输出: Dog::bark
}
```

实现父类/祖类的接口

有时，抽象类实现某接口时，可以不实现接口中的方法，而是留给子类实现。

代码块

```
classes! {
    // 抽象的接口定义
    abstract class Eat {
        pub fn eat(&self);
    }

    // 抽象类定义，但并不实现接口中的方法，而是留给子类实现
    abstract class Animal implements Eat {
        pub fn new() -> Self {}
        // `Animal`是抽象类，因此可以不实现其父接口类的`eat`方法，但其非抽象子类必须实现。
        // pub override fn eat(&self) {}
```

```
    }

    class Dog extends Animal {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }
        // 实现父类`Animal`的接口`Eat`中的方法`eat`
        // 实现直接父类的接口，也可以写作`override fn <Super as Eat>::eat`
        // pub override fn <Super as Eat>::eat(&self)
        pub override fn <Animal as Eat>::eat(&self) {
            println!("Dog::eat");
        }
    }
}

fn main() {
    let dog = Dog::new();
    dog.eat(); // 输出: Dog::eat
}
```

向上/向下转型

代码块

```
classes! {
    class I {}
    class A {}
    class B extends A {}
    mixin M on B {}
    class C extends B with M {}
}
```

以上述几个类为例，提供了下列方法可以在几个类之间转型：

代码块

```
// 向上转型
impl CRc<C> {
    // 向祖类转型（优先使用）
    pub fn as_superclass<A>(&self) -> &A;
    pub fn to_superclass<A>(&self) -> A;
    pub fn into_superclass<A>(self) -> A;

    // 万能向上转型（父类、父接口、父混入类）
    pub fn to_supertype<S>(&self) -> S;
```

```
    pub fn into_supertype<S>(self) -> S;
}

// 向下转型
impl CRc<C> {
    // 向子孙类转型（优先使用）
    // - 转换失败时panic
    pub fn as_subclass<D>(&self) -> &D;
    pub fn to_subclass<D>(&self) -> D;
    pub fn into_subclass<D>(self) -> D;
    // - 转换失败时返回None
    pub fn try_as_subclass<D>(&self) -> Option<&D>;
    pub fn try_to_subclass<D>(&self) -> Option<D>;
    pub fn try_into_subclass<D>(self) -> Option<D>;
    // - 转换失败时为未定义行为
    pub unsafe fn as_subclass_unchecked<D>(&self) -> &D;
    pub unsafe fn to_subclass_unchecked<D>(&self) -> D;
    pub unsafe fn into_subclass_unchecked<D>(self) -> D;

    // 万能向下转型
    pub fn to_subtype<D>(&self) -> D;
    pub fn into_subtype<D>(self) -> D;
    pub fn try_to_subtype<D>(&self) -> Option<D>;
    pub fn try_into_subtype<D>(self) -> Option<D>;
 }
```

其余转型方法

代码块

```
impl CRc<C> {
    // 向直接父类转型
    pub fn as_super(&self) -> &CRc<C::Super>;
    pub fn to_super(&self) -> CRc<C::Super>;
    pub fn into_super(self) -> CRc<C::Super>;
    // 向直接父接口转型
    pub fn to_impl<I>(&self) -> I;
    pub fn into_impl<I>(self) -> I;
    // 向mixin () 转型
    pub fn to_mixin<M>(self) -> M;

    // 向父祖类的直接父接口类转型（A是Self的祖类，且A直接实现了接口类I）
    // self.upcast::<A, I>()等价于self.as_superclass::<A>().to_impl::<I>()
    pub fn upcast<A, I>(&self) -> A;

    // 向子接口类的子孙类转型（B是Self的直接子接口类即B直接实现了Self接口，D是B的子孙类
```

```
    // - 转换失败时panic
    pub fn downcast<B, D>(&self) -> D;
    // - 转换失败时返回None
    pub fn try_downcast<B, D>(&self) -> Option<D>;
    // - 转换失败时为未定义行为
    pub unsafe fn downcast_unchecked(&self) -> D;
}

impl CRc<M> {
    // 向mixin上的on列表或impl列表转型
    pub fn mixin_to_impl<B>(&self) -> B;
    // 向mixin实例的子类转型
    pub fn mixin_downcast<C>(&self) -> C;
    pub fn mixin_try_downcast<C>(&self) -> Option<C>;
    pub unsafe fn mixin_downcast_unchecked<C>(&self) -> C;
}
```

代码块

```
classes! {
    abstract class Eat {
        pub fn eat();
    }

    class Alice implements Eat {
        pub fn new() -> Self { Self {} }
        pub override fn eat(&self) {
            println!("Alice::eat");
        }
    }

    class Bob extends Alice {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }
        pub override fn <Super as Eat>::eat(&self) {
            println!("Bob::eat");
        }
    }

    class Carol extends Bob {
        pub fn new() -> Self {
            Self { super: Super::new() }
        }
        pub override fn <Alice as Eat>::eat(&self) {
            println!("Carol::eat");
```

```rust
        }
    }
}

fn main() {
    // 创建一个Carol类的实例
    let carol: CRc<Carol> = Carol::new();
    carol.eat(); // 输出: Carol::eat
    // 使用`as_super`可以将`carol`向上转为其直接父类类型
    let bob: CRc<Bob> = carol.as_super();
    bob.eat(); // 输出: Carol::eat
    // 使用`as_superclass`可以将`carol`向上转为其父/祖类类型
    let alice: CRc<Alice> = carol.as_superclass();
    alice.eat(); // 输出: Carol::eat
    // 使用`upcast`可以将`carol`向上转型为祖类`Alice`的实现的接口`Eat`类型
    let eat: CRc<Eat> = carol.upcast::<CRc<Alice>, _>();
    // 上述语句等价于
    // let eat: CRc<Eat> = alice.to_impl();
    eat.eat();  // 输出: Carol::eat

    // 使用`downcast`可以将`eat`经`Alice`类型向下转型为`Bob`类型
    let bob: CRc<Bob> = eat.downcast::<CRc<Alice>, _>();
}
```